

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
import matplotlib.pyplot as plt
```

```
def gini_impurity(labels):
    """Calculate Gini impurity for a list of class labels."""
    labels = np.array(labels)
    n = len(labels)
    if n == 0: return 0
    classes = np.unique(labels)
    gini = 1.0
    for c in classes:
        p = np.sum(labels == c) / n # proportion of class c
        gini -= p ** 2
    return gini
```

```
print('Pure node (all same) :', gini_impurity([1,1,1,1,1])) # 0.00
print('50-50 split :', gini_impurity([0,0,0,1,1,1])) # 0.50
print('70-30 split :', gini_impurity([0,0,0,0,0,0,0,1,1,1])) # 0.42
print('90-10 split :', gini_impurity([0]*9 + [1])) # 0.18
```

```
Pure node (all same) : 0.0
50-50 split : 0.5
70-30 split : 0.42000000000000004
90-10 split : 0.17999999999999999
```

```
def weighted_gini(left_labels, right_labels):
    """Gini after a split - weighted by group size."""
    n_total = len(left_labels) + len(right_labels)
    w_left = len(left_labels) / n_total
    w_right = len(right_labels) / n_total
    return w_left * gini_impurity(left_labels) + w_right * gini_impurity(right_labels)
```

```
left_A = [0,0,0,0,1]
right_A = [0,1,1,1,1]
```

```
left_B = [0,0,1,1,0]
right_B = [0,1,0,1,1]
```

```
print(f'Weighted Gini - Glucose split: {weighted_gini(left_A, right_A):.3f}')
print(f'Weighted Gini - Age split : {weighted_gini(left_B, right_B):.3f}')
print('Tree picks the split with LOWER weighted Gini → Glucose split is better!')
```

```
Weighted Gini - Glucose split: 0.320
Weighted Gini - Age split : 0.480
Tree picks the split with LOWER weighted Gini → Glucose split is better!
```

```
from sklearn.datasets import make_classification
from sklearn.tree import export_text
X, y = make_classification(n_samples=200, n_features=4, random_state=42)
tree = DecisionTreeClassifier(max_depth=2, random_state=42)
tree.fit(X, y)
print(export_text(tree, feature_names=['F1', 'F2', 'F3', 'F4']))
```

```
|--- F1 <= -0.15
|   |--- F3 <= -0.44
|   |   |--- class: 0
|   |--- F3 > -0.44
|   |   |--- class: 0
|--- F1 > -0.15
|   |--- F3 <= 0.63
|   |   |--- class: 1
|   |--- F3 > 0.63
|   |   |--- class: 0
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
```

```
x,y=make_classification(n_samples=200,n_features=7,random_state=42)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
depths=range(1,11)
train_acc,test_acc=[],[]
```

```

train_acc, test_acc = [], []
for d in depths:
    tree=DecisionTreeClassifier(max_depth=d,random_state=42)
    tree.fit(x_train,y_train)
    train_acc.append(accuracy_score(y_train,tree.predict(x_train)))
    test_acc.append(accuracy_score(y_test,tree.predict(x_test)))

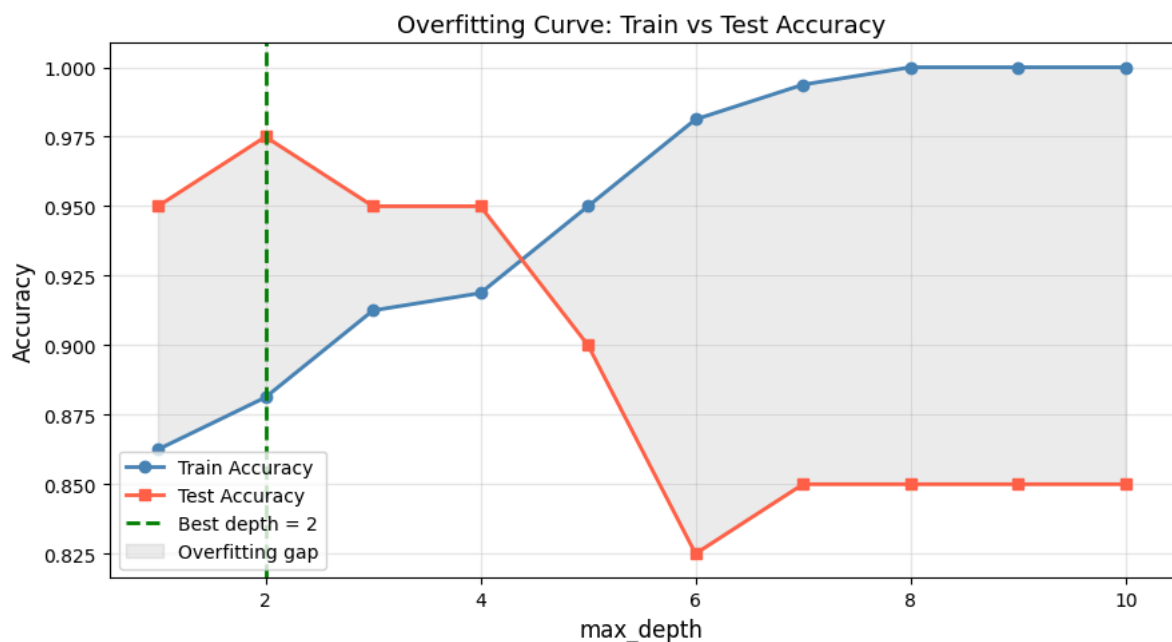
```

```
best_depth=depths[test_acc.index(max(test_acc))]
```

```

plt.figure(figsize=(9, 5))
plt.plot(depths, train_acc, 'o-', color='steelblue', lw=2, label='Train Accuracy')
plt.plot(depths, test_acc, 's-', color='tomato', lw=2, label='Test Accuracy')
plt.axvline(x=best_depth, color='green', linestyle='--', lw=2,
            label=f'Best depth = {best_depth}')
plt.fill_between(depths,
                train_acc, test_acc,
                alpha=0.15, color='gray', label='Overfitting gap')
plt.xlabel('max_depth', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('Overfitting Curve: Train vs Test Accuracy', fontsize=13)
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
print(f'Best test accuracy {max(test_acc):.2%} at depth={best_depth}')

```



Best test accuracy 97.50% at depth=2

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

```

```

configs=[
    {'max_depth':None},
    {'max_depth':5},
    {'max_depth':5,'min_samples_split':10},
    {'max_depth':5,'min_samples_leaf':5},
    {'max_depth':5,'max_features':'sqrt'},
]

```

```

print(f'{{Config':45s}} {{CV Score(mean+/-std):20s}}')
print('-'*60)

```

```

Config                                CV Score(mean+/-std)
-----

```

```

for cfg in configs:
    tree=DecisionTreeClassifier(**cfg,random_state=42)
    scores=cross_val_score(tree,x,y,cv=5,scoring='accuracy')
    print(f'{{str(cfg):45s}} {{scores.mean():.3f}} +/- {{scores.std():.3f}}')

```

```

{'max_depth': None}                0.795 +/- 0.062
{'max_depth': 5}                   0.810 +/- 0.044
{'max_depth': 5, 'min_samples_split': 10}  0.795 +/- 0.048

```

<code>{'max_depth': 5, 'min_samples_leaf': 5}</code>	0.820 +/- 0.040
<code>{'max_depth': 5, 'max_features': 'sqrt'}</code>	0.660 +/- 0.060